



CSEE 4290: Computer Architecture

Top Level SCC – Spring 2025 – Group 5

[https://github.com/Herring-UGACSEE-4290-F25/Group-5/tree/main/Proj2\(SCC\)/TestCases](https://github.com/Herring-UGACSEE-4290-F25/Group-5/tree/main/Proj2(SCC)/TestCases)

Completed

Instruction Fetch

Decode

Execute

Register File

Fix Unconditional Branch Bubble

Implement uCode MUL/S

Create Testbench

Write Unique Test Case

Run All Class Test Cases

To-Do

Final documentation and code clean-up + comments

Group Responsibilities

Test Bench: Jordan

Instruction Fetch: Ricky

Instruction Decode: Matthew

Execute: Nathaniel

uCode MUL/S: Everybody

Schedule

ID/IF: 10-03

Execute 3 Instructions/Branch Imm: 10-10

Execute Complete: 10-20

Testcases: 10-24

SCC Complete: 10-29

Testcase Plan

- Check writability to R15
- Check every branch condition
- Check ORS and OR
- Check MUL (Data Register)
- Doublecheck if MOVT and MOVF works in regular program as well

ISA Map

- MOV: Bubble, 1, 2, 3, 5
- MOVT: 2, 5, 6
- MOVF: 2
- LOAD: Bubble, 1, 2, 5
- STOR: Bubble, 1, 2, 3, 5, 6, 7

- MUL: Multiply (uCode Inclass Check), 1, 5, 6
- MULS: Multiply (uCode Inclass Check), 1, 6
- ADD: Bubble, 1, 2, 5, 6
- ADDS: 3, 5
- SUB: Bubble, 1, 2, 3, 5
- SUBS: Bubble, 2, 3, 5, 6
- AND: Multiply (Test Case), 2, 3, 5, 6
- ANDS: Multiply (Test Case), 5, 6
- OR: 3, 5, 6
- ORS: 3, 5
- XOR: 2, 3, 6
- XORS: 3
- NOT: Multiply (Test Case), 5
- B: Bubble Sort: 1, 3, 5, 6
- B.cond: Bubble Sort: 1, 2, 3, 5, 6
- BR: 2, 6
- HALT: ALL
- LSL: Multiply (Test Case), 1, 2, 5, 6
- LSR: Multiply (Test Case), 1, 2, 5, 6
- CLR: Bubble Sort, 6
- SET: 2, 6
- NOP: 1

Testbench

Our testbench compares the data memory output of our SCC against the output of the emulator to see if they match.

```
`timescale 1ns/1ns
```

```
module scc_tb;
```

```
localparam MEM_BYTES = 1<<16;
```

```
reg clk_tb = 1;
```

```
reg rst_tb = 1;
```

```
reg clk_en_tb = 1;
```

```
wire halt_f_tb;
```

```
wire [1:0] err_bits_tb;
```

```
wire [31:0] instruction_memory_v_tb;
```

```
wire [31:0] data_memory_in_v_tb;
```

```
scc_f25_top dut (
```

```
    .clk(clk_tb),
```

```
    .rst(rst_tb),
```

```
    .clk_en(clk_en_tb),
```

```
    .halt_f(halt_f_tb),
```

```
    .err_bits(err_bits_tb),
```

```
    .instruction_memory_v(instruction_memory_v_tb),
```

```
    .data_memory_in_v(data_memory_in_v_tb)
```

```
);
```

```
always #5 clk_tb = ~clk_tb;
```

```
// release reset after 3 clock cycles
```

```
initial begin
```

```

repeat (3) @(posedge clk_tb);

rst_tb = 1'bo;

end

reg [7:0] dut_mem [0:MEM_BYTES-1];


initial begin

    $dumpfile("dump.vcd");

    $dumpvars(0, scc_tb);

end


// compare memory dump to CSV file
initial begin : COMPARE

    integer fd,fd2;

    integer matched,matched2;

    integer line_num;

    reg [255:0] line,line2;

    reg [31:0] addr, value,addr2,value2;

    reg [31:0] got_word;


    // wait until HALT from the SCC

    wait(halt_f_tb);

```

```
$display("Apollo has Landed!");

@(posedge clk_tb); // wait a clock after HALT to let $writememh finish in
instruction_and_data

// load SCC memory dump file

//$readmemh("scc_out.txt", dut_mem);

fd2 = $fopen("scc_out.txt", "r");
if (fd2 == 0) begin
    $display("ERROR: could not open scc_out.txt");
    $finish;
end

// open CSV file
fd = $fopen("dataoutput.csv", "r");
if (fd == 0) begin
    $display("ERROR: could not open dataoutput.csv");
    $finish;
end

matched = $fgets(line, fd); // read and ignore top line in dataoutput file
```

```
matched2 = $fgets(line2,fd2);
```

```
matched2 = $sscanf(line2, "0x%h , 0x%h", addr2, value2);
```

```
if (matched2 != 2) matched2 = $sscanf(line2, "0x%h,0x%h", addr2, value2);
```

```
if (matched2 != 2) matched2 = $sscanf(line2, "%h , %h", addr2, value2);
```

```
if (matched2 != 2) matched2 = $sscanf(line2, "%h,%h", addr2, value2);
```

```
line_num = 1;
```

```
while (!$feof(fd)) begin
```

```
    line_num = line_num + 1;
```

```
    if ($fgets(line, fd)) begin
```

```
        // tolerate optional spaces and optional 0x prefixes
```

```
        matched = $sscanf(line, "0x%h , 0x%h", addr, value);
```

```
        if (matched != 2) matched = $sscanf(line, "0x%h,0x%h", addr, value);
```

```
        if (matched != 2) matched = $sscanf(line, "%h , %h", addr, value);
```

```
        if (matched != 2) matched = $sscanf(line, "%h,%h", addr, value);
```

```
        if (matched == 2) begin
```

```
            // bounds check
```

```
            while(addr!=addr2) begin
```

```
                if($fgets(line2,fd2)) begin
```

```

    matched2 = $sscanf(line2, "0x%h , 0x%h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "0x%h,0x%h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "%h , %h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "%h,%h", addr2, value2);

end

end

matched2 = $sscanf(line2, "0x%h , 0x%h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "0x%h,0x%h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "%h , %h", addr2, value2);

    if (matched2 != 2) matched2 = $sscanf(line2, "%h,%h", addr2, value2);

if ((addr+3) >= MEM_BYTES) begin

    $display("FAIL @ line %0d: addr 0x%08h out of range", line_num, addr);

    $finish;

end

// assemble word: MSB at lowest address (matches your mem read logic)

got_word = {dut_mem[addr+0], dut_mem[addr+1], dut_mem[addr+2],
dut_mem[addr+3]};

if (value2 != value) begin

    $display("FAIL @ line %0d: addr=0x%08h got=0x%08h exp=0x%08h",

        line_num, addr, value2, value);

    $finish;

end

else begin

```

```

        $display("Passed @ line %0d: addr=0x%08h got=0x%08h exp=0x%08h",
            line_num, addr, value2, value);
    end
end

end

end

$display("PASS: all CSV memory values match DUT dump.");
$fclose(fd);
$fclose(fd2);
$finish;
End

endmodule

```

Multiply Testcase

The test case used for this checkpoint was our multiply instruction but ran without microcode or setting the flags back to their original state. The algorithm is explained in more detail in the Multiply section.

START:

```
ADD Ro, R14, #10000; //Operand 1
```

```
ADD R1, R14, #5; //Operand 2
```

OPER1:

SUBS R0, R0, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER1NEG;

OPER2:

SUBS R1, R1, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER2NEG;

OPERATION:

ANDS R3, R1, #1; //LSB of multiplier

B.EQ OPERATIONNEXT;

ADD R2, R2, R0;

OPERATIONNEXT:

LSL R0, R0, #1;

LSR R1, R1, #1;

SUBS R1, R1, #0x0; //See if multiplier is 0

B.EQ OUTPUT;

B OPERATION;

OPER1NEG:

NOT R0, R0; //2's complement

ADD R0, R0, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPER2;

OPER2NEG:

NOT R1, R1; //2's complement

ADD R1, R1, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPERATION;

OUTPUT:

ANDS R7, R5, #1;

HALT

Bubble Sort Testcase

ORG #0x400;

FCB #0x5;

FCB #0x3;

FCB #0x7;

FCB #0x1;

FCB #0x4;

FCB #0x9;

FCB #0x6;

FCB #0x8;

FCB #0x2;

FCB #0xA;

ORG #0x0;

setup:

MOV R0, #0x400;

MOV R4, #0x400; R4 placeholder for array

MOV R5, #0x9; length of array

MOV R7, #0x0; loop count value

mainloop:

CMP R5, #0x0;

B.eq exit; exit if length is zero

LOAD R1, R4; initial pointer to array

iter:

CMP R5, R7;

B.eq iterexit; exit pass if count = length

B compare;

compreturn:

ADD R4, R4, #0x4; increment array pointer by 4

ADD R7, R7, #0x1; increment count

LOAD R1, R4; set value from pointer

B iter;

iterexit:

SUB R4, R4, R4;

MOV R4, #0x400; reset address to current first value

SUB R5, R5, #0x1; subtract one from length

CLR R7; reset count to 0

B mainloop;

compare:

LOAD R2, R4, #0x4; load second value to compare

CMP R2, R1; compare two values

B.mi swap; swap if $b - a < 0$

B comreturn;

swap:

STOR R1, R4, #0x4; #store value one in address two

STOR R2, R4; #store value two in address one

B comreturn;

exit:

HALT;

Multiply Algorithm

- First checks if the multiplicand is 0 or a negative with a SUBS #0. If the value is 0, then it branches to the end. If the value is negative, then it branches to make a 2's complement of the multiplicand and adds 1 to a counter that keeps track of how many negative operators the MULT had (R5). If the value is neither a 0 nor negative, then it moves on to check the multiplier with the exact same process.
- Once both operators are positive, the algorithm then checks the LSB of the multiplier to see if it is a 1 by using the ANDS instruction. If it is, then the value of the multiplicand is added to the product register (R2).
- The multiplicand is then shifted left by 1 and the multiplier right by 1, and the process above is repeated until the multiplier is equal to 0.
- The algorithm then used another ANDS on R5 to check how many negative operands the MULT had and determines whether the output should be negative or not. If the output should be negative, then it converts R2 to its 2's complement and goes on to set the proper flags. If the output is positive, then it branches to set the flags.
- The flags are always returned to their original state (Using a series of ANDS to check against the flags in R4 which was set by MOVF). The flags will then be newly set by the final value of the multiply depending on if it was a MUL or MULS instruction.
- **NOTE: The setting flags section of the code as since been replaced with the newly added SAVF instruction**

START:

ADD R0, R14, r0; //Operand 1

ADD R1, R14, #0; //Operand 2

MOVF R4; //Original Flag

OPER1:

SUBS R0, R0, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER1NEG;

OPER2:

SUBS R1, R1, #0; //Check Neg

B.EQ OUTPUT;

B.MI OPER2NEG;

OPERATION:

ANDS R3, R1, #1; //LSB of multiplier

B.EQ OPERATIONNEXT;

ADD R2, R2, R0;

OPERATIONNEXT:

LSL R0, R0, #1;

LSR R1, R1, #1;

SUBS R1, R1, #0x0; //See if multiplier is 0

B.EQ OUTPUT;

B OPERATION;

OPER1NEG:

NOT R0, R0; //2's complement

ADD R0, R0, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPER2;

OPER2NEG:

NOT R1, R1; //2's complement

ADD R1, R1, #1;

ADD R5, R5, #1; //Add to Neg counter

B OPERATION;

OUTPUT:

ANDS R7, R5, #1;

B.EQ FLAGSHELL;

NEGATIVEOUT:

NOT R2, R2; //2's complement

ADD R2, R2, #1;

FLAGSHELL:

ANDS R6,R4,#8; //Masking for the N flag and checking if it's on

B.EQ NEG;

POS:

ANDS R6,R4,#4; //Masking for the Z flag

B.NE ZERO;

ANDS R6,R4,#2; //Masking for the C flag

B.NE CARRY;

ANDS R6,R4,#1; //Masking for the V flag

```
B.NE OVER;

MOV32 R0,#1;

MOV32 R1,#0;

ADDS R6,R0,R1; //Setting no flag

B FINALDONE;
```

OVER:

```
MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#0x80000000;

SUBS R6,R0,R1; //Setting V flag

B FINALDONE;
```

CARRY:

```
ANDS R6,R4,#1; //Masking for CV flag

B.NE CARRYV;

MOV32 R0,#1;

MOV32 R1,#0;

SUBS R6,R0,R1; //Setting C flag
```

CARRYV:

```
MOV32 R0,#0xFFFFFFFF;

MOV32 R1,#0x80000000;

ADDS R6,R0,R1; //Setting CV flags
```

ZERO:

```
ANDS R6,R4,#2; //Masking for ZC flags

B.NE ZEROC;
```

```
MOV32 R0,#0;
MOV32 R1,#0;
ADDS R6,R0,R1; //Setting Z flag
B FINALDONE;
```

ZEROC:

```
ANDS R6,R4,#1; //Masking for ZCV flags
B.NE ZEROVC;
MOV32 R0,#1;
MOV32 R1,#0xFFFFFFFF;
ADDS R6,R0,R1; //Setting ZC flags
B FINALDONE;
```

ZEROVC:

```
MOV32 R0,#0x80000000;
MOV32 R1,#0x80000000;
ADDS R6,R0,R1; //Setting ZCV flags
B FINALDONE;
```

NEG:

```
ANDS R6,R4,#2; //Masking for NC flags
B.NE NEGC;
ANDS R6,R4,#1; //Masking for NV flags
B.NE NEGV;
MOV32 R0,#0xFFFFFFFF;
MOV32 R1,#1;
```

```
ADDS R6,R0,R1; //Setting N flag
```

```
B FINALDONE;
```

NEGV:

```
MOV32 R0,#1;
```

```
MOV32 R1,#0xFFFFFFFF;
```

```
ADDS R6,R0,R1; //Setting NV flags
```

```
B FINALDONE;
```

NEGC:

```
MOV32 R0,#0xFFFFFFFF;
```

```
MOV32 R1,#1;
```

```
ADDS R6,R0,R1; //Setting NC flags
```

```
B FINALDONE;
```

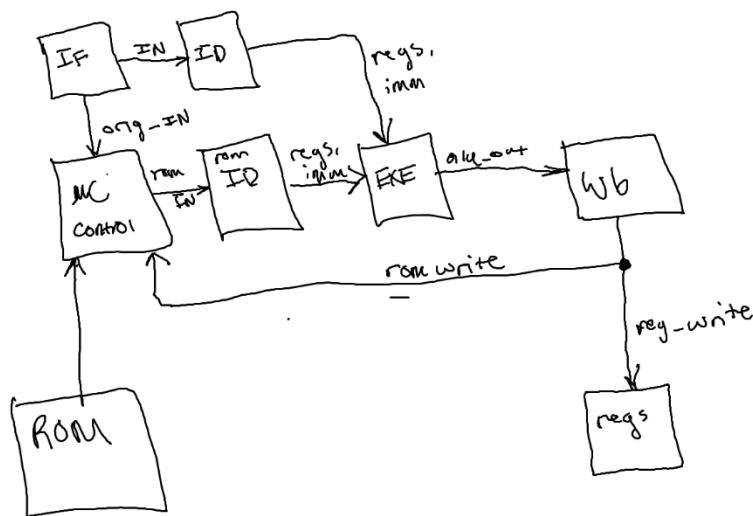
FINALDONE:

```
ADD R2, R2, #0;
```

```
HALT;
```

uCode Handler

Our uCode handler has its own IF stage and set of registers. Once a multiply instruction is detected, the main program's IF stage will halt and the uCode will take over. The uCode's IF stage acts exactly like the main program's except it will inject/change some ROM instructions to allow the uCode to import values from the main registers into the ROM registers, to export the output value from the ROM register to the main destination register (And set flags if MUL/I S was used), and to set the immediate values as an operand if a MULI/S was used. Both the uCode handler and the main program share the same ID, EXE, and WB stages. The uCode knows the multiply is over once it sees a HALT instruction in the ROM.



AFTER MC MUL:

- Send ADD R_d, R₂, #0;
- Send instructions for flags

~~have wb in~~
~~MC so wb~~

Floating Point Addition Testcase

- The main algorithm for the operation is to first check the two operands for the floating-point special cases of 0, +/- inf, and NaN by using ANDS (Check 0 flag) and CMP (Check 0 flag) against a bit mask of 0x7F800000 (Checks for inf).
- If the first operand is an inf, the program will then check the second operand to see if it is the opposite inf. If so, then the result is 0. If not, then the result will be the inf or NaN. If an operand is 0, then the result will be the other operand.
- The program then uses 2 masks and AND operations to store the exponent and mantissa of each operand into 2 registers (4 in total). An OR operation is used to add the implicit MSB of 1 back into the mantissas for future calculations. An LSR is also used to shift the exponents by 23 bits so that the values can be easily added and subbed from.
- The program then does a compare between the exponents of the two operands to see which one is larger. The goal is to shift the smaller of the two operands so that the exponent matches that of the larger exponent. A branch conditional is used to move to the area of the program that shifts either operand 1 or 2. The smaller of the two operands has their mantissa LSR by 1 and their exponent ADD by 1. Another compare is then performed to check if the exponents are equal. If the exponents are not, then a branch is used to loop back. If the exponents are equal, then the program branches to check for negatives.
- A CMP against 0 is used on the original operands to check if the sign is negative or not for both operands. If an N flag is detected, a branch is taken and the 2's

complement is performed on the corresponding mantissa before branching back to continue the program.

- The two mantissas are then added together with an ADDS and the N flag is again checked to see if the resulting mantissa is negative. If so, then the 2's complement is taken for the resulting mantissa and a mask is placed into a register with a signed N bit for later.
- The program then checks if the resulting floating-point value is normalized. First, to check if a right shift is needed for normalization, a CMP is done between the bit mask for the mantissa (with the implicit 1 included) and the resulting mantissa. If the resulting mantissa is larger than the mask, then the program LSR the mantissa by 1 and ADD the exponent by 1 before looping back to compare again. Else, the program moves on to check if a left shift is required.
- To check if a left shift is required for normalization, a CMP is done between the resulting mantissa and a mask of just the implicit 1 of the mantissa (where the mantissa consists of all 0s except for the implicit MSB of 1). If the resulting mantissa is smaller than the mask (N flag == 1), then the resulting mantissa is LSL by 1 and the exponent is SUB by 1 before branching back to check for the shift again. Else, the program branches to recompile the results into a single floating-point value.
- To recompile, a mask is first used again to remove the implicit 1 from the mantissa and the exponent is LSR 23 bits back into its original position. 2 OR instructions are then used to combine the exponent, mantissa, and signed bit into the results register.
- In our final test case, instead of nonconditional branches, the SAVF instruction is used alongside others to create the necessary flags to test nearly every conditional branch. The program is also looped 6 times to test 6 different operands for adds.

```
ORG    #0x400;
```

```
FCB    #0xC1200000;op1 num 1
```

```
FCB    #0xC1200000;op1 num 2
```

```
FCB    #0;      op1 output
```

```
FCB    #0xC1200000;op2 num 1
```

FCB #0x40400000;op2 num 2

FCB #0; op2 output

FCB #0x00000000;op3 num 1

FCB #0xC1200000;op3 num 2

FCB #0; op3 output

FCB #0x7F800000;op4 num 1

FCB #0xFF800000;op4 num 2

FCB #0; op4 output

FCB #0x7F810000;op5 num 1

FCB #0xFF800000;op5 num 2

FCB #0; op5 output

FCB #0x40200000;op6 num 1

FCB #0x3ff00000;op6 num 2

FCB #0; op6 output

ORG #0x0;

START:

MOV R13,#6; //counter for how many times to loop

MOV R11,#0x400; //mem location for data

;//R0 operand 1
;//R1 operand 2
;//R2 Mask
;//R3 operand 1 exponent
;//R4 operand 2 exponent
;//R5 operand 1 mantissa
;//R6 operand 2 mantissa
;//R7 Mantissa calc
;//R8 result sign
;//R9 Implicit 1 of mantissa
;//r10 burner flag reg
;//R12 Result
;//0x40400000: 3 0x41400000: 12 0xC1200000: -10

LoadNum:

MOV R2, #0;
MOV R3, #0;
MOV R4, #0;
MOV R5, #0;
MOV R6, #0;
MOV R7, #0;
MOV R8, #0;
MOV32 R9, #0x00800000;

MOV R12, #0;

LOAD R0, r11; //op1

LOAD R1, r11,#4; //op2

ors r5,r5,r5

b.ne #0xFFFF;

CHKZERO1:

MOV32 R2, #0x7F800000; //Masking exponents for o

ANDS R3, R0, R2; //Masking operand 1

B.EQ ZEROADD1;

CHKZERO2:

ANDS R4, R1, R2; //Masking operand 2

B.EQ ZEROADD2;

CHKINF1:

CMP R3, R2;

B.EQ NAN1;

CHKINF2:

CMP R4, R2;

B.EQ NAN2;

GRBMAN:

MOV32 R2, #0x007FFFFFFF; //Mask for mantissa

AND R5, R0, R2;

AND R6, R1, R2;

OR R5, R5, R9; //Add implicit 1

OR R6, R6, R9; //Add implicit 1

LSR R3, R3, #23; //Shift exponents

LSR R4, R4, #23;

MOV32 R2, #0x00FFFFFF; //Mask for implicit 1 mantissa

CHKOP1G:

CMP R3, R4; //Check which exponent is larger

B.EQ CHKNEG1;

B.MI OP2G;

OP1G:

ADD R4, R4, #1; //Add op2 exponent

LSR R6, R6, #1; //Shift right op2 mantissa

CMP R3, R4;

B.NE OP1G;

mov r10, #4; //nzcv

SAVF r10; //was a normal branch using this to check cases checked on 2

B.LE CHKNEG1;

OP2G:

ADD R3, R3, #1; //Add op1 exponent

LSR R5, R5, #1; //Shift right op1 mantissa

CMP R3, R4;

B.NE OP2G;

CHKNEG1:

CMP R0, #0; //Check op1 is neg

B.MI OP1COMP;

CHKNEG2:

CMP R1, #0; //Check op2 is neg

B.MI OP2COMP;

mov r10,#0; //nzcv

SAVF r10; //was a normal branch using this to check cases checked on 1

B.GE OPADD;

OP1COMP:

NOT R5, R5; //2's complement

ADD R5,R5, #1;

mov r10,#4; //nzcv

SAVF r10; //was a normal branch using this to check cases checked on 1

B.LS CHKNEG2;

OP2COMP:

NOT R6, R6; //2's complement

ADD R6,R6, #1;

OPADD:

ADDS R7, R5, R6; //Add the results and check if negative

B.PL SHIFTRIGHT;

RESCOMPL:

NOT R7, R7;

ADD R7,R7, #1;

MOV32 R8, #0x80000000; //Set signed as neg

SHIFTRIGHT:

CMP R2, R7;

B.PL SHIFLEFT;

LSR R7,R7, #1; //Shift right by 1

ADD R3,R3, #1; //Add 1 to exponent

mov r10,#2;//nzc

SAVF r10; //was a normal branch using this to check cases checked on 1

B.HI SHIFTRIGHT;

SHIFLEFT:

CMP R7, R9; //Compare mantissa to implicit 1 value

B.PL SETRESULT;

LSL R7,R7, #1; //Shift left by 1

SUB R3,R3, #1; //Subtract 1 from exponent

mov r10,#0;//nzc

SAVF r10; //was a normal branch using this to check cases checked on 2

B.VC SHIFLEFT;

ZEROADD1:

Add R12, R1,#0;

mov r10,#1;//nzc

SAVF r10; //was a normal branch using this to check cases checked on 3

B.VS DONE;

ZEROADD2:

Add R12, R0,#0;

B DONE;

NAN1:

CMP R4, R2;

B.EQ NANBoth;

Add R12, R0,#0;

B DONE;

NAN2:

Add R12, R1,#0;

B DONE;

NANBoth:

MOV32 R2, #0x007FFFFFFF;

ANDS R5, R0, R2;

b.ne NAN;

ANDS R6, R1, R2;

b.ne NAN;

cmp r0,r1;

b.eq NAN2;

mov r12,#0;

```
mov r10,#0;//nzcw
```

```
SAVF r10; //was a normal branch using this to check cases  checked on 4
```

```
B.LO DONE;
```

NAN:

```
MOV32 r12, #0x7fc00000;
```

```
b DONE;
```

SETRESULT:

```
MOV32 R2, #0x007FFFFFFF; //Mask for mantissa
```

```
AND R7, R7, R2;
```

```
LSL R3,R3, #23; //Shift exponent back
```

```
OR R12, R8, R3;
```

```
OR R12, R12, R7;
```

DONE:

```
STOR R12,r11,#8;
```

```
add r11,r11,#12;
```

```
sub r13,r13,#1;
```

```
MULS r14,r13,r13; //using this instead of subs to check an instruct no one had checked
```

```
b.ne LoadNum
```

```
HALT;
```

(Class Testcases) Bugs

- Scc_output.txt was in the incorrect format so other testbenches could not read (fixed formatting)
- \$fclose() was never used in memory file when writing the output so the testbenches could not read (fixed by including a close)
- \$fclose() was also forgotten in the testbench and crashed (fixed by including a close)
- SET was only setting 1 bit to 1 instead of 32 bits (fixed by setting 32 bits)
- 2 MULs could not be used back-to-back because the flag for preventing catch was not updated in time (fixed by updating the value when the program counter ticks)
- BR did not work because we forgot to add it (fixed by supporting the instruction)
- LOAD was broken because we added a random IF statement in preparation for BR and forgot about it (fixed by removing the statement)
- LOAD was also broken because somehow branch conditionals were added to be checked for a memory operation in EXE (fixed by removing the branch statement)
- Our test case was broken because we accidentally reverted to part of a previous iteration where we used MOV register to register instead of an ADD instruction (fixed by using the ADD)
- Our test case couldn't compile because we accidentally deleted a ; before a comment (fixed by adding a ;)
- Failed Group 2's test because the assembler has a bug where it will skip over a memory address if the value is set to 0x00000000 and the LOAD instruction ends up grabbing XXXXXXXX instead of 0 (fixed by manually adding in the missing memory address value at the location)